Sparse Quantum Simulation

CSC591/ECE592 Project Proposal

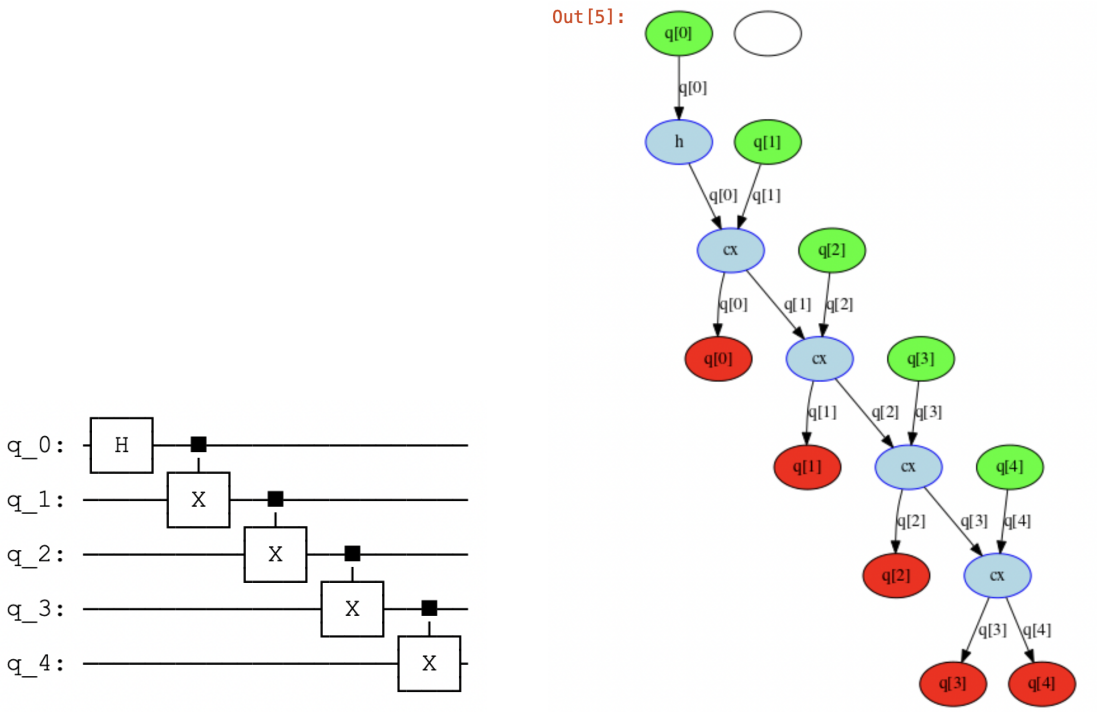
Blake Burgstahler[1] and Srikar Chundury[1]

[1]*North Carolina State University*

(Dated: October 18, 2022)

I. INTRODUCTION

Tensors can be thought of as multi-dimensional maps or functions. For example, a 2 dimensional tensor would be a matrix. A graph with tensors as vertices and "tensor methods" as edges is called a tensor network. Tensor methods could be contractions (matrix multiplication in 2D), decompositions/factorizations (PCA [6], SVD [2] in 2D) and more. Tensor networks have useful implications in applied mathematics, chemistry, physics, machine learning and many other fields.



(a) Circuit Representation

(b) Tensor network representation

Figure 1: Two ways of representing the same circuit

Quantum circuits can be modelled as tensor networks not only to study and understand them better, but also to efficiently simulate them on classical computers. Fig. 1 shows one such way to adapt a circuit to a tensor network.

Efficient simulation of quantum circuits is important for two main reasons:

1. Quantum hardware is costly, and state preparation takes time which makes it infeasible to run multiple attempts while coming up new quantum algorithms,
2. Today's quantum computers suffer from problems like decoherence and noise which can hinder algorithm accuracies.

Classical simulation of quantum circuits is memory bound, namely, with every additional qubit the memory requirement doubles [3]. Hence, efficient contraction algorithms that "evolve" through the tensor network become crucial for better performance.

Today's tensor network based quantum simulations involve a few steps:

1. Representation of a given quantum circuit as a tensor-network.
2. Local optimizations like decompositions to sub-networks.
3. Finding optimal contraction path [1].
4. Pair-wise contraction of tensors to arrive at a final state in case of a state simulator, or a unitary matrix in case of a unitary simulator.

II. PROBLEM DESCRIPTION

The tensors in quantum tensor networks are quite sparse and follow specific patterns. Today's libraries that leverage tensor networks like quimb [4], qiskit [5] represent all tensors in dense formats.

We aim to produce a numerical library which will leverage the sparsity introduced in the above-described simulation process and help achieve performance gains with respect to storage in memory and overall simulation time. In this way, we can both model sub-circuits as well as the overall circuit by considering their (hopefully) sparse representation as a unitary matrix – or tensor products of unitary matrices. We expect to then be able to pass our sparse data structure directly to qiskit using the 'unitary' and 'UnitaryGate' functions included with qiskit.

III. PROPOSED TIMELINE

Table I shows the proposed schedule for this project.

Week of...	Work
Oct 18	- Initial setup of project spaces (Git, overleaf, communication methods, etc.)
	- Completion and submission of project summary/proposal
Oct 24	- Further Literature review and commit to sparse tensor representation
	- Start building library functions
Oct 31	- Kronecker product, Dot product, Matrix-Vector, and Matrix-Matrix Product complete
	- Package code as installable library
Nov 7	- Ensure integration with *Aer.simulator.** (statevector)
Nov 14	- Ensure integration with qiskit and run simulations on Supermarq/CertiQ, compare metrics

Table I: Tentative Schedule

IV. DELIVERABLES

- Installable package which implements the aforementioned contraction kernels. Each optimized for quantum specific sparsity.
  - Github repo: `https://github.ncsu.edu/tnqs/sparse-tnqs`
  - Git io page: `https://pages.github.ncsu.edu/tnqs/sparse-tnqs/`
- Presentation to chronicle approach/results.
- Report expanding on presentation, if time allows.

---

[1] Daniel G. a. Smith and Johnnie Gray. opt_einsum - a python package for optimizing contraction order for einsum-like expressions. *Journal of Open Source Software*, 3(26):753, 2018.

[2] Steve Brunton. Singular value decomposition (svd): Overview, 2020.

[3] Mircosoft Corporation. Operations on multiple qubits, 2022.

[4] Johnnie Gray. quimb: a python library for quantum information and many-body calculations. *Journal of Open Source Software*, 3(29):819, 2018.

[5] IBM. Learn quantum computation using qiskit, 2021.

[6] J. Nathan Kutz. Principal component analysis (pca), 2016.